Part 22: Feature Engineering

Part 22: Feature Engineering

Financial Data Science II Updated March 16, 2026

Data rarely arrive in a form that is immediately suitable for predictive analysis. Feature engineering refers to the construction of variables from the observed data that may be informative for a forecasting or decision problem.

In finance this step is especially important. Financial signals are weak, relationships change over time, and even small errors in timing can invalidate an analysis. Accordingly, the construction of features should be treated as part of the modelling problem itself, rather than as a routine preprocessing step.

Here we consider predictive features built from historical market data. A later treatment could consider factor models, long-short portfolio construction, and the relation between factor exposures and tradeable signals.

Exercise: Give three examples of raw data encountered in finance and explain how each could be transformed into a useful feature for prediction.

Basic Terms

We use the following terms throughout.

A **feature** (or **predictor**) is a variable used to predict some future quantity. A **target** (or **response**) is the quantity to be forecast. A **forecast horizon** is the time between the observation of the feature and the measurement of the target.

For daily equity data, a common target is a 5-day forward return. For limit order book data, a common target is the next movement of the mid-price. These are distinct problems and generally call for distinct feature sets.

Two additional distinctions are important.

1. **Clock time:** observations are aggregated to regular intervals such as one minute or one day.
2. **Event time:** observations are indexed by market events such as order submissions, cancellations, or trades.

Finally, **lookahead bias** occurs when a feature contains information that would not have been available at the prediction time. This is one form of **data leakage**, and it can make an otherwise poor procedure appear successful in backtesting.

Exercise: Suppose that you wish to predict a stock's return over the next 5 trading days. What information is valid to use on day t? What information would constitute leakage?

Why Feature Engineering Matters in Finance

Feature engineering matters in any predictive problem, but several aspects of financial data make it especially important here.

1. Financial data are noisy.
2. Relationships are often nonstationary.
3. Timing is critical. A feature that is valid at the daily level may be invalid at the intraday level.
4. Useful predictors often arise from economic or market-structure considerations rather than from generic transformations alone.

Thus, a modest model with carefully constructed features can outperform a more sophisticated model fitted to weak or misaligned inputs.

Example Prediction Tasks

To fix ideas, we consider two examples in this Part.

1. **Daily equity prediction:** use market and accounting data to predict a forward return over several days.
2. **Limit order book prediction:** use LOBSTER message and order book data to predict short-horizon mid-price direction.

The first example emphasizes feature families commonly used in equity prediction. The second emphasizes market microstructure and the distinction between event-time data and fixed-interval bars.

Example A: Daily Equity Data

We begin with a standard daily equity example using `yfinance`. The objective is not to argue that the particular features below are optimal, but to illustrate a systematic procedure for constructing and validating daily predictors.

The feature families considered here are:

1. returns and log returns
2. momentum over multiple horizons
3. rolling volatility
4. drawdown
5. valuation ratios such as P/E and P/B when available

```
import warnings
warnings.filterwarnings('ignore')

from pathlib import Path

import numpy as np
import pandas as pd
import yfinance as yf

from sklearn.base import clone
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.linear_model import LinearRegression, LogisticRegression, Ridge
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    roc_auc_score,
)
from sklearn.model_selection import TimeSeriesSplit
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

try:
    import xgboost as xgb
    HAS_XGB = True
except Exception:
    HAS_XGB = False
```

```
# Example ticker and date range
TICKER = 'AAPL'
START = '2016-01-01'
END = '2026-03-01'

px = yf.download(TICKER, start=START, end=END, auto_adjust=False, progress=False)
```

```
if isinstance(px.columns, pd.MultiIndex):
    px.columns = [str(level0).lower().replace(' ', '_') for level0, _ in px.columns]
else:
    px.columns = [str(col).lower().replace(' ', '_') for col in px.columns]

px = px.rename(columns={'adj_close': 'adj_close'})
px = px.dropna()

px.head()
```

	adj_close	close	high	low	open	volume
Date						
2016-01-04	23.730946	26.337500	26.342501	25.500000	25.652500	270597600
2016-01-05	23.136259	25.677500	26.462500	25.602501	26.437500	223164000
2016-01-06	22.683500	25.174999	25.592501	24.967501	25.139999	273829600
2016-01-07	21.726149	24.112499	25.032499	24.107500	24.670000	324377600
2016-01-08	21.841034	24.240000	24.777500	24.190001	24.637501	283192000

Exercise: Inspect the downloaded data. Which columns are observed directly from the market, and which are already transformed?

```
# Construct a small set of price-based features
feat = pd.DataFrame(index=px.index)
feat['ret_1d'] = px['adj_close'].pct_change()
feat['log_ret_1d'] = np.log(px['adj_close'] / px['adj_close'].shift(1))

for horizon in [5, 21, 63, 126]:
    feat[f'mom_{horizon}d'] = px['adj_close'].pct_change(horizon)
    feat[f'vol_{horizon}d'] = feat['ret_1d'].rolling(horizon).std()

rolling_max = px['adj_close'].cummax()
feat['drawdown'] = px['adj_close'] / rolling_max - 1

feat = feat.dropna()
feat.head()
```

	ret_1d	log_ret_1d	mom_5d	vol_5d	mom_21d	vol_21d	mom_63d	vol_6
Date								
2016-07-05	-0.009385	-0.009430	0.032052	0.010179	-0.029922	0.011551	-0.129693	0.0146
2016-07-06	0.005684	0.005668	0.020729	0.008373	-0.031431	0.011498	-0.133817	0.0146
2016-07-07	0.004292	0.004283	0.016313	0.008004	-0.031202	0.011503	-0.110705	0.0144
2016-07-08	0.007713	0.007683	0.011297	0.006742	-0.022842	0.011676	-0.104835	0.0144
2016-07-11	0.003103	0.003098	0.011367	0.006744	-0.026794	0.011566	-0.105022	0.0144

Accounting-Based Features

A second class of predictors is based on accounting information. For illustration, we form quarterly estimates of earnings per share and book value per share, and then convert them into P/E and P/B ratios after aligning them to the daily index.

As in all such constructions, timing is the main concern. In production work, one should use point-in-time accounting data rather than revised statements.

```
def get_financial_table(ticker, attr):
    obj = getattr(ticker, attr, None)
    return obj if isinstance(obj, pd.DataFrame) else pd.DataFrame()


def get_share_series(ticker):
    shares = None
    try:
        shares = ticker.get_shares_full()
    except Exception:
        try:
            shares = ticker.get_shares()
        except Exception:
            shares = None

    if isinstance(shares, pd.DataFrame) and shares.shape[1] > 0:
        shares = shares.iloc[:, 0]

    if shares is not None and hasattr(shares, 'index'):
        shares = shares[~shares.index.duplicated(keep='last')]
        shares.index = pd.to_datetime(shares.index, errors='coerce')
        try:
            shares.index = shares.index.tz_localize(None)
```

```
        except TypeError:
            pass
    return shares


def as_series(x):
    if isinstance(x, pd.DataFrame):
        if x.shape[1] == 0:
            return pd.Series(dtype=float)
        return x.iloc[:, 0]
    return x


def build_fundamental_features(ticker, index, price_series):
    income_q = get_financial_table(ticker, 'quarterly_income_stmt')
    if income_q.empty:
        income_q = get_financial_table(ticker, 'income_stmt')

    balance_q = get_financial_table(ticker, 'quarterly_balance_sheet')
    if balance_q.empty:
        balance_q = get_financial_table(ticker, 'balance_sheet')

    shares = get_share_series(ticker)
    fund = pd.DataFrame(index=index)
    price_series = as_series(price_series).reindex(index)

    if shares is None or len(shares) == 0:
        return fund

    if not income_q.empty:
        income_q = income_q.T
        income_q.index = pd.to_datetime(income_q.index, errors='coerce')
        net_income_col = next((c for c in income_q.columns if 'Net Income' in c), None)
        if net_income_col is not None:
            eps = income_q[net_income_col] / shares.reindex(income_q.index, method='nea
            eps = as_series(eps)
            fund['eps_q'] = eps.replace([np.inf, -np.inf], np.nan).reindex(index, metho

    if not balance_q.empty:
        balance_q = balance_q.T
        balance_q.index = pd.to_datetime(balance_q.index, errors='coerce')
        equity_col = next(
            (c for c in balance_q.columns if 'Total Stockholder Equity' in c or 'Total
            None,
        )
        if equity_col is not None:
            bvps = balance_q[equity_col] / shares.reindex(balance_q.index, method='near
            bvps = as_series(bvps)
            fund['bvps_q'] = bvps.replace([np.inf, -np.inf], np.nan).reindex(index, met

    if 'eps_q' in fund.columns:
        fund['pe'] = price_series / as_series(fund['eps_q'])
    if 'bvps_q' in fund.columns:
        fund['pb'] = price_series / as_series(fund['bvps_q'])

    return fund
```

```
tk = yf.Ticker(TICKER)
fund = build_fundamental_features(tk, px.index, px['adj_close'])
```

```
features_daily = feat.join(fund[['pe', 'pb']] if {'pe', 'pb'}.issubset(fund.columns) el
if features_daily.dropna().empty:
    features_daily = feat.copy()
else:
    features_daily = features_daily.dropna()

features_daily.head()
```

	ret_1d	log_ret_1d	mom_5d	vol_5d	mom_21d	vol_21d	mom_63d	vol_
Date								
2024-10-01	-0.029142	-0.029575	-0.005102	0.018809	-0.012183	0.015860	0.028156	0.014
2024-10-02	0.002520	0.002517	0.001811	0.018739	0.018001	0.014633	0.024791	0.014
2024-10-03	-0.004895	-0.004907	-0.008131	0.018661	0.021825	0.014533	-0.001806	0.014
2024-10-04	0.005007	0.004995	-0.004346	0.018875	0.019876	0.014501	-0.003325	0.014
2024-10-07	-0.022531	-0.022789	-0.048541	0.015258	0.003940	0.015305	-0.029445	0.014

Constructing the Target

We now define a 5-day forward return target. This requires shifting the response forward in time. If one instead uses same-day returns, or otherwise allows future prices to enter the predictor set, the resulting evaluation is invalid.

Random shuffling is likewise inappropriate for time series. Temporal order should be preserved both in model fitting and in validation.

```
TARGET_HORIZON = 5

features_daily['target_fwd_ret_5d'] = (
    px['adj_close'].reindex(features_daily.index).shift(-TARGET_HORIZON)
    / px['adj_close'].reindex(features_daily.index)
    - 1
)
features_daily = features_daily.dropna()

X_daily = features_daily.drop(columns=['target_fwd_ret_5d'])
y_daily = features_daily['target_fwd_ret_5d']

X_daily.head(), y_daily.head()
```

```
(              ret_1d  log_ret_1d    mom_5d    vol_5d   mom_21d   vol_21d  \
Date
2024-10-01 -0.029142   -0.029575 -0.005102  0.018809 -0.012183  0.015860
2024-10-02  0.002520    0.002517  0.001811  0.018739  0.018001  0.014633
2024-10-03 -0.004895   -0.004907 -0.008131  0.018661  0.021825  0.014533
2024-10-04  0.005007    0.004995 -0.004346  0.018875  0.019876  0.014501
2024-10-07 -0.022531   -0.022789 -0.048541  0.015258  0.003940  0.015305

             mom_63d   vol_63d  mom_126d  vol_126d  drawdown         pe  \
Date
2024-10-01  0.028156  0.014754  0.343159  0.016215 -0.035551  93.504825
2024-10-02  0.024791  0.014741  0.340115  0.016213 -0.033121  93.740439
2024-10-03 -0.001806  0.014505  0.340112  0.016213 -0.037854  93.281611
2024-10-04 -0.003325  0.014495  0.340786  0.016214 -0.033036  93.748705
2024-10-07 -0.029445  0.014762  0.319369  0.016346 -0.054822  91.636469

                   pb
Date
2024-10-01  50.885741
2024-10-02  51.013963
2024-10-03  50.764267
2024-10-04  51.018461
2024-10-07  49.868973  ,
Date
2024-10-01   -0.001945
2024-10-02    0.012170
2024-10-03    0.014933
2024-10-04    0.003307
2024-10-07    0.043349
Name: target_fwd_ret_5d, dtype: float64)
```

Exercise: Why is the following split inappropriate in this setting?

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X_daily, y_daily,
test_size=0.2)
```

Temporal Validation

For the daily regression problem, we use a rolling time-series split. The reported metrics are:

1. RMSE
2. MAE
3. R^2
4. sign accuracy

The final metric is crude, but may still be useful in finance when the sign of the forecast matters more than its exact magnitude.

```
models_daily = {
    'Linear': LinearRegression(),
    'Ridge': Ridge(alpha=1.0),
    'RandomForest': RandomForestRegressor(
        n_estimators=200,
        random_state=42,
        n_jobs=-1,
        min_samples_leaf=5,
    ),
}

if HAS_XGB:
    models_daily['XGBoost'] = xgb.XGBRegressor(
        objective='reg:squarederror',
        random_state=42,
        n_estimators=300,
        learning_rate=0.05,
        max_depth=4,
        subsample=0.8,
        colsample_bytree=0.8,
```

```
    )

splitter = TimeSeriesSplit(n_splits=5)
rows = []

for name, estimator in models_daily.items():
    fold_rows = []
    for train_idx, test_idx in splitter.split(X_daily):
        X_train = X_daily.iloc[train_idx]
        X_test = X_daily.iloc[test_idx]
        y_train = y_daily.iloc[train_idx]
        y_test = y_daily.iloc[test_idx]

        model = clone(estimator)
        model.fit(X_train, y_train)
        pred = model.predict(X_test)

        fold_rows.append(
            {
                'rmse': np.sqrt(mean_squared_error(y_test, pred)),
                'mae': mean_absolute_error(y_test, pred),
                'r2': r2_score(y_test, pred),
                'sign_accuracy': (np.sign(pred) == np.sign(y_test)).mean(),
            }
        )

    fold_summary = pd.DataFrame(fold_rows).mean()
    rows.append({'model': name, **fold_summary.to_dict()})

results_daily = pd.DataFrame(rows).sort_values('rmse').reset_index(drop=True)
results_daily
```

	model	rmse	mae	r2	sign_accuracy
0	XGBoost	0.048159	0.039229	-0.363163	0.513725
1	RandomForest	0.049459	0.040835	-0.431507	0.494118
2	Ridge	0.049682	0.040750	-0.652814	0.482353
3	Linear	0.103437	0.085713	-9.678264	0.450980

Example B: LOBSTER Data

We now turn to a limit order book example using a LOBSTER sample for AAPL. Unlike the daily example above, these data are event-driven: each row corresponds to a market event together with a synchronized snapshot of the book.

The feature families considered here are:

1. spread and mid-price
2. level-1 and multi-level depth imbalance
3. order flow imbalance (OFI)
4. microprice deviation
5. message intensity

Because event-time data are not equally spaced, we will later aggregate them to 1-second bars using `floor(time)`.

```
LOBSTER_DIR = Path('/Users/harold/4. RA work/FACTOR_TRAINING_CHI/lobster_samples/AAPL_2
msg_path = LOBSTER_DIR / 'AAPL_2012-06-21_34200000_57600000_message_5.csv'
ob_path = LOBSTER_DIR / 'AAPL_2012-06-21_34200000_57600000_orderbook_5.csv'

msg = pd.read_csv(
    msg_path,
    header=None,
    names=['time', 'type', 'order_id', 'size', 'price', 'direction'],
)

levels = 5
ob_cols = []
for level in range(1, levels + 1):
    ob_cols += [
        f'ask_price_{level}',
        f'ask_size_{level}',
        f'bid_price_{level}',
        f'bid_size_{level}',
    ]
ob = pd.read_csv(ob_path, header=None, names=ob_cols)

for col in [c for c in ob.columns if 'price' in c] + ['price']:
    if col in msg.columns:
        msg[col] = msg[col] / 10000.0
    if col in ob.columns:
        ob[col] = ob[col] / 10000.0

msg.head()
```

	time	type	order_id	size	price	direction
0	34200.004241	1	16113575	18	585.33	1
1	34200.004261	1	16113584	18	585.32	1
2	34200.004447	1	16113594	18	585.31	1
3	34200.025552	1	16120456	18	585.91	-1
4	34200.025580	1	16120480	18	585.92	-1

Event-Level Features

The best bid and best ask determine the top of the book. From these quantities one may compute the mid-price, the spread, and several imbalance measures.

Order flow imbalance (OFI) is intended to summarize changes in demand and supply at the book. The single-level version focuses on the best quotes, while a multi-level version accumulates information across several depth levels.

```
def compute_ofi_one_level(bid_price, bid_size, ask_price, ask_size):
    bid_prev = bid_price.shift(1)
    ask_prev = ask_price.shift(1)
    bid_size_prev = bid_size.shift(1)
    ask_size_prev = ask_size.shift(1)

    delta_w = np.select(
        [bid_price > bid_prev, bid_price == bid_prev, bid_price < bid_prev],
        [bid_size, bid_size - bid_size_prev, -bid_size_prev],
        default=0.0,
    )
    delta_v = np.select(
        [ask_price < ask_prev, ask_price == ask_prev, ask_price > ask_prev],
        [ask_size, ask_size - ask_size_prev, -ask_size_prev],
        default=0.0,
    )
    out = pd.Series(delta_w - delta_v, index=bid_price.index, dtype=float)
    out.iloc[0] = 0.0
    return out


feat_lob = pd.DataFrame(index=msg.index)
feat_lob['time'] = msg['time']
feat_lob['time_str'] = pd.to_timedelta(feat_lob['time'], unit='s')
feat_lob['mid'] = (ob['ask_price_1'] + ob['bid_price_1']) / 2.0
feat_lob['spread'] = ob['ask_price_1'] - ob['bid_price_1']
feat_lob['spread_bps'] = 10000.0 * feat_lob['spread'] / feat_lob['mid']
feat_lob['imbalance_l1'] = (
    (ob['bid_size_1'] - ob['ask_size_1']) / (ob['bid_size_1'] + ob['ask_size_1'])
)

bid_depth = sum(ob[f'bid_size_{level}'] for level in range(1, levels + 1))
ask_depth = sum(ob[f'ask_size_{level}'] for level in range(1, levels + 1))
feat_lob['imbalance_l5'] = (bid_depth - ask_depth) / (bid_depth + ask_depth)
```

```
feat_lob['microprice'] = (
    ob['ask_price_1'] * ob['bid_size_1'] + ob['bid_price_1'] * ob['ask_size_1']
) / (ob['bid_size_1'] + ob['ask_size_1'])
feat_lob['microprice_dev_bp'] = 10000.0 * (feat_lob['microprice'] - feat_lob['mid']) /
feat_lob['signed_size'] = msg['direction'] * msg['size']

for level in range(1, levels + 1):
    feat_lob[f'ofi_{level}'] = compute_ofi_one_level(
        ob[f'bid_price_{level}'],
        ob[f'bid_size_{level}'],
        ob[f'ask_price_{level}'],
        ob[f'ask_size_{level}'],
    )

feat_lob['ofi_l1'] = feat_lob['ofi_1']
feat_lob['ofi_l5'] = sum(feat_lob[f'ofi_{level}'] for level in range(1, levels + 1))

feat_lob[['time', 'time_str', 'mid', 'spread_bps', 'imbalance_l1', 'imbalance_l5', 'ofi
```

	time	time_str	mid	spread_bps	imbalance_l1	imbalance_l5	
0	34200.004241	0 days 09:30:00.004241176	585.635	10.416044	-0.834862	-0.561216	
1	34200.004261	0 days 09:30:00.004260640	585.635	10.416044	-0.834862	-0.544715	
2	34200.004447	0 days 09:30:00.004447484	585.635	10.416044	-0.834862	-0.639344	
3	34200.025552	0 days 09:30:00.025551909	585.620	9.904033	0.000000	-0.629104	
4	34200.025580	0 days 09:30:00.025579546	585.620	9.904033	0.000000	-0.505325	

Resampling in Event Time

The LOBSTER data are not equally spaced in time, so we construct fixed 1-second bars. To avoid lookahead, each observation is assigned to `floor(time)` and then aggregated within that second.

For quote-like quantities, we retain the **last** value in the second. For flow-like quantities such as OFI or signed size, we use a **sum**.

```
feat_lob['time_floor'] = np.floor(feat_lob['time']).astype(int)

bars_lob = (
    feat_lob.groupby('time_floor', sort=True)
    .agg(
        time=('time', 'last'),
        time_str=('time_str', 'last'),
        mid=('mid', 'last'),
```

```
        spread_bps=('spread_bps', 'last'),
        imbalance_l1=('imbalance_l1', 'last'),
        imbalance_l5=('imbalance_l5', 'last'),
        microprice_dev_bp=('microprice_dev_bp', 'last'),
        ofi_l1=('ofi_l1', 'sum'),
        ofi_l5=('ofi_l5', 'sum'),
        signed_size=('signed_size', 'sum'),
        msg_count=('mid', 'size'),
    )
    .reset_index()
)

bars_lob['next_mid'] = bars_lob['mid'].shift(-1)
bars_lob['next_mid_ret_bp'] = 10000.0 * (bars_lob['next_mid'] - bars_lob['mid']) / bars
bars_lob['target_up_1s'] = (bars_lob['next_mid'] > bars_lob['mid']).astype(int)
bars_lob = bars_lob.dropna().reset_index(drop=True)

bars_lob.head()
```

	time_floor	time	time_str	mid	spread_bps	imbalance_l1	imb
0	34200	34200.918221	0 days 09:30:00.918220904	585.805	2.219168	0.000000	
1	34201	34201.840852	0 days 09:30:01.840851751	585.620	5.122776	0.000000	
2	34202	34202.756909	0 days 09:30:02.756909029	585.575	3.586219	0.694915	
3	34203	34203.954770	0 days 09:30:03.954770435	585.580	4.440042	-0.694915	
4	34204	34204.894471	0 days 09:30:04.894471165	585.590	3.073823	-0.960784	

Temporal Validation for LOB Features

This second problem is a classification task. Because the target classes are imbalanced, raw accuracy can be misleading. We therefore report:

1. accuracy
2. balanced accuracy
3. AUC

For numerical stability, extreme feature values are clipped using quantiles estimated on the training portion of each split.

```
feature_cols_lob = [
    'spread_bps',
    'imbalance_l1',
    'imbalance_l5',
    'microprice_dev_bp',
```

```
    'ofi_l1',
    'ofi_l5',
    'signed_size',
    'msg_count',
]

X_lob = bars_lob[feature_cols_lob].replace([np.inf, -np.inf], np.nan).dropna().copy()
y_lob = bars_lob.loc[X_lob.index, 'target_up_1s']

models_lob = {
    'Logistic': Pipeline(
        [
            ('scale', StandardScaler()),
            ('model', LogisticRegression(max_iter=1000, class_weight='balanced')),
        ]
    ),
    'RandomForest': RandomForestClassifier(
        n_estimators=200,
        max_depth=6,
        min_samples_leaf=20,
        random_state=42,
        n_jobs=-1,
        class_weight='balanced_subsample',
    ),
}

if HAS_XGB:
    models_lob['XGBoost'] = xgb.XGBClassifier(
        n_estimators=300,
        max_depth=4,
        learning_rate=0.05,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42,
        eval_metric='logloss',
    )

rows = []
for name, estimator in models_lob.items():
    fold_rows = []
    for train_idx, test_idx in TimeSeriesSplit(n_splits=5).split(X_lob):
        X_train = X_lob.iloc[train_idx].copy()
        X_test = X_lob.iloc[test_idx].copy()
        y_train = y_lob.iloc[train_idx]
        y_test = y_lob.iloc[test_idx]

        for col in feature_cols_lob:
            lo = X_train[col].quantile(0.01)
            hi = X_train[col].quantile(0.99)
            X_train[col] = X_train[col].clip(lower=lo, upper=hi)
            X_test[col] = X_test[col].clip(lower=lo, upper=hi)

        model = clone(estimator)
        model.fit(X_train, y_train)
        prob = model.predict_proba(X_test)[:, 1]
        pred = (prob >= 0.5).astype(int)

        fold_rows.append(
            {
                'accuracy': accuracy_score(y_test, pred),
```

```
                'balanced_accuracy': balanced_accuracy_score(y_test, pred),
                'auc': roc_auc_score(y_test, prob),
            }
        )

    fold_summary = pd.DataFrame(fold_rows).mean()
    rows.append({'model': name, **fold_summary.to_dict()})

results_lob = pd.DataFrame(rows).sort_values('auc', ascending=False).reset_index(drop=T
results_lob
```

	model	accuracy	balanced_accuracy	auc
0	RandomForest	0.614967	0.552966	0.579828
1	Logistic	0.621847	0.540694	0.563199
2	XGBoost	0.717260	0.512146	0.561027

What Makes a Feature Useful?

A useful feature need not have a large marginal correlation with the target. More generally, a feature is attractive when it satisfies several criteria at once:

1. it carries predictive information
2. it remains reasonably stable across time or regimes
3. it is interpretable
4. it can be computed in real time
5. it survives practical constraints such as trading cost, latency, or limited data availability

For this reason, feature engineering is better viewed as a modelling discipline than as a list of formulas.

Exercises

Exercise 1: Add rolling moving-average ratios to the daily feature set. Re-run the daily evaluation and compare the results to those obtained from the baseline set of return, momentum, volatility, and drawdown features.

Exercise 2: In the daily example, explain why accounting variables can create timing problems even when they appear to be aligned by date.

Exercise 3: In the LOBSTER example, compare `imbalance_l1` and `ofi_l1`. Which is easier to interpret economically? Which appears more useful empirically?

Exercise 4: Suppose that the LOBSTER data are aggregated using the ceiling of the event time rather than the floor. Explain why this may create leakage.

References

1. Cont, Kukanov, and Stoikov (2013), *The Price Impact of Order Book Events*.
2. Gould, Porter, Williams, McDonald, Fenn, and Howison (2013), *Limit Order Books*.
3. LOBSTER documentation and sample files.
4. Little and Rubin, *Statistical Analysis with Missing Data*.
5. Lopez de Prado, *Advances in Financial Machine Learning*.